

HOWTO: build the oculomotor circuit, the way the heading-direction circuit was built

2026-08-11

HOWTO: build the oculomotor circuit, the way the heading-direction circuit was built

Audience: whoever owns the oculomotor cell list (the colleague) plus whoever wires it into the model. Companion to `HOWTO_add_zebrafish_circuit.md`, which covers the generic case; this file is the concrete oculomotor recipe traced off what actually produced the 917-cell HD circuit.

0. First, correct one assumption

"the list of neurons and neuron types and E/I was given by my colleagues and we must have queried neuPrint with a script?"

Half true, and the half that matters is the other one. There are **two** independent routes in this repo, and the circuit behind the paper came from the second:

	route 1 — neuPrint query	route 2 — colleagues' pickle
script	<code>fetch_zebrafish_connectivity_-HD_IPN12.py</code>	<code>build_connectome_HD_IPN12_943_-from_pickle.py</code>
source	<code>neuprint-fish2.janelia.org</code> , dataset <code>fish2</code> , live	<code>IPN_sortedData_060826.pkl</code> from the colleagues
cell list	a hardcoded python list of 30 fish2 type strings	a hardcoded python list of 34 type strings, filtering the pickle's 1854 cells
edge weight	<code>weight = synapse count</code>	<code>adjacency_matrix_size = synapse contact area</code>
produced	<code>zebrafish_connectome_HD_IPN12/</code> — 839 cells	<code>zebrafish_connectome_HD_IPN-917/</code> — 917 cells, 30,851 edges
used by	the older 839-cell figures	every figure in <code>zebrafish_paper.tex</code>

So for the circuit that matters, neuPrint was queried for exactly **one** thing: the soma XYZ coordinates (`soma_fish2.csv`, used only for the 3-D anatomy renders). The cell identities, the cell types, the L/R hemisphere and the connectivity matrix all came out of the colleagues' pickle.

And E/I came from neither. There is no Dale annotation on the fish2 server. The signs are assigned *by cell-type prefix, in our code*, at `connectome_loaders.py:493`:

```
_ZHD_INH_PREFIXES = ("IPNd", "IPNds", "IPN12_a", "IPN12_b", "IPNc")
```

Every cell whose type starts with one of those has its **outgoing column** forced negative (`J[:, col] = -inh_amplify * |J[:, col]|`); everything else stays positive. That single line is the entire E/I model, and it

encodes a literature claim (Petrucco et al. 2023: the $r1\pi$ /dIPN cells are GABAergic), not a measurement. For oculomotor this line is the thing your colleague has to supply the content of.

1. What a "circuit" actually is

Two CSVs on disk plus one registry function. Nothing else.

```
figures/zebrafish/zebrafish_connectome_OCULO_<N>/
  neurons.csv bodyId, type, instance, side, somaLocationX/Y/Z [, angle]
  connections.csv bodyId_pre, bodyId_post, weight
  meta.json provenance (source file, n_neurons, n_edges, conventions)
```

`neurons.csv` row order is not the model's neuron order — the loader re-sorts (readout pool first, then afferents). `connections.csv` is a sparse edge list; `weight > 0` always, the sign is applied later from the type prefix.

2. What we need from the colleague — the input checklist

Nothing below can be guessed from the EM data. Get all five before writing code.

1. **The cell-type list.** The exact type strings to keep, verbatim as they

appear in their file. For HD this was 34 strings; two candidate types (IPN20, IPN26) were explicitly dropped on their request. Ask for the drop list too, not just the keep list.

1. **The E/I assignment per type.** Which types are inhibitory. For the

oculomotor integrator the expected answer is structured — the Goldman-lab model negates the DO/MO populations and zeroes ABD/vSPN/IBN/axial (see `connectome_loaders.py:1010`) — so ask whether they want the same partition or a fresh one.

1. **The functional split.** Which types are the **readout/integrator pool**

(the HD equivalent: the bump ring, 700 cells) and which are **afferents** (where input is injected). For oculomotor: which cells carry the saccadic velocity command in, and which cells' rates represent eye position out.

1. **The ordering variable.** HD uses a per-cell preferred-heading angle

(`IPN_angles`) so the matrix plots as a ring. Oculomotor is a line attractor, not a ring — the natural analogue is the per-cell integration time constant or the position/velocity coefficient. Ask what they have per cell; if nothing, fall back to soma position and say so in `meta.json`.

1. **The source file itself**, with the key names spelled out. For HD the

pickle carried `Type`, `Hemi`, `NeuronIDs`, `adjacency_matrix_size`, `IPN_angles` — and one field (`zb`) that was misaligned and had to be discarded. Expect one such trap.

Also settle **edge orientation** explicitly: is `A[i, j]` pre→post or post→pre? For HD this was confirmed empirically — the afferents send 64 % of their output to the ring, which only holds under one of the two readings. Do the same check; do not assume.

3. The procedure

Step 1 — write the converter

Copy `build_connectome_HD_IPN12_943_from_pickle.py` to `figures/zebrafish/build_connectome_OCULO_from_<source>.py` and change four things: the `CELLTYPES` list (input #1), the source path, the output dir, and the per-cell ordering column (input #4). Run once:

```
python figures/zebrafish/build_connectome_OCULO_from_<source>.py
```

Verify before moving on — this is the stage where a silent transpose or an off-by-one in the type/angle alignment costs a week:

- N matches the number the colleague expects, exactly;
- `meta.json` records the source file and the orientation convention;
- afferent cells' outgoing edges land mostly on the integrator pool

(the orientation check);

- the type histogram matches theirs type-by-type, not just in total.

Step 2 — teach the loader the new taxonomy

In `connectome_loaders.py`, add an oculomotor sibling of `_zhd_category` / `load_zebrafish_hd_connectome`. Do **not** extend the HD one in place — its prefix rules are order-sensitive (IPNdS must match before IPNd) and the 917 circuit must not shift.

Three constants to define, mirroring lines 476–493:

```
_ZOC_READOUT_PREFIXES = (...) # the integrator pool (HD analogue: bump ring)
_ZOC_AFFERENT_PREFIXES = (...) # velocity-command input cells
_ZOC_INH_PREFIXES = (...) # input #2 --the entire E/I model
```

A type not matched by any of these is **dropped**, silently. Assert the kept count equals N from step 1.

Step 3 — register the circuit

In `circuits.py`, add `_build_zebrafish_oculomotor(...)` + `_register_zebrafish_oculomotor()`, and call the latter from `_discover_circuits` (next to line 445). Declare the subpops the model will gather:

```
subpops = {
    "bump": <integrator pool indices>, # keep the key name
    "afferent_<name>_L": ..., "afferent_<name>_R": ...,
    "readout": <cells the decoder reads>,
}
```

Set `inh_amplify` and `spectral_target` deliberately and record them in **provenance**. The 917 circuit uses `inh_amplify=1.0` (keep relative E/I magnitudes, because the weights are contact areas) and `spectral_target=0.9`. A count-weighted matrix would want the HD-839 setting (`inh_amplify=5.0`).

Step 4 — point a config at it

```
circuit:
  name: zebrafish_OCULO_<N>_v1
simulation:
  connconstr_datapath: figures/zebrafish/zebrafish_connectome_OCULO_<N>
```

Start from `zebrafish_hd_si_ipn_917_v1_selfmotion_rotation.yaml` — it is the minimal trained baseline (one target, `task_targets: [rotation]`).

Step 5 — the seams that will fight you

The IO path is hard-bound to the HD afferent taxonomy in five places. If the oculomotor afferents are not expressible as four L/R subpops, you hit **Procedure B** of `HOWTO_add_zebrafish_circuit.md` — seams (i)–(v), including the hardcoded `n_input = 3` and the positional `r[... , :n_dipn]` readout slice. That is a real refactor, not a config change. Read §B2 of that file before promising a date.

4. Naming: do not collide with the existing biomodel

zebrafish_oculomotor **already exists** as a separate biomodel — 609 neurons, Goldman-lab `.mat` connectome at `papers/Code_NN/Code_NN/Data/Figure5/goldman_data`, a linear integrator ODE (Beiran & Litwin-Kumar 2023 Fig 5), 14 configs in `GraphData/config/zebrafish_oculomotor`. It solves the *inverse* problem (simulate dynamics $\rightarrow$ recover the connectome). It has never been run in this environment: `graphs_data/zebrafish_oculomotor` is empty and there is no `log/zebrafish_oculomotor`.

The circuit described in this document is the *opposite* direction — connectome-constrained task training, the HD lineage. Keep it under the **zebrafish** biomodel with a circuit name like **zebrafish_OCULO_<N>_v1**, so the two do not share a config namespace or a log tree.